

Artificial
Intelligence



J.CS712 ХИЙМЭЛ ОЮУН УХААН

Боловсруулсан багш: Дэд проф., Др., Д. Золзаяа

Мэйл хаяг: zolzaya@must.edu.mn

КУ-ны салбар

ШУТИС МХТС, 2021

Хайлт

function GENERAL-SEARCH(*problem*, QUEUING-FN) **returns** a solution, or failure

nodes $\leftarrow$ MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[*problem*]))

loop do

if *nodes* is empty **then return** failure

node $\leftarrow$ REMOVE-FRONT(*nodes*)

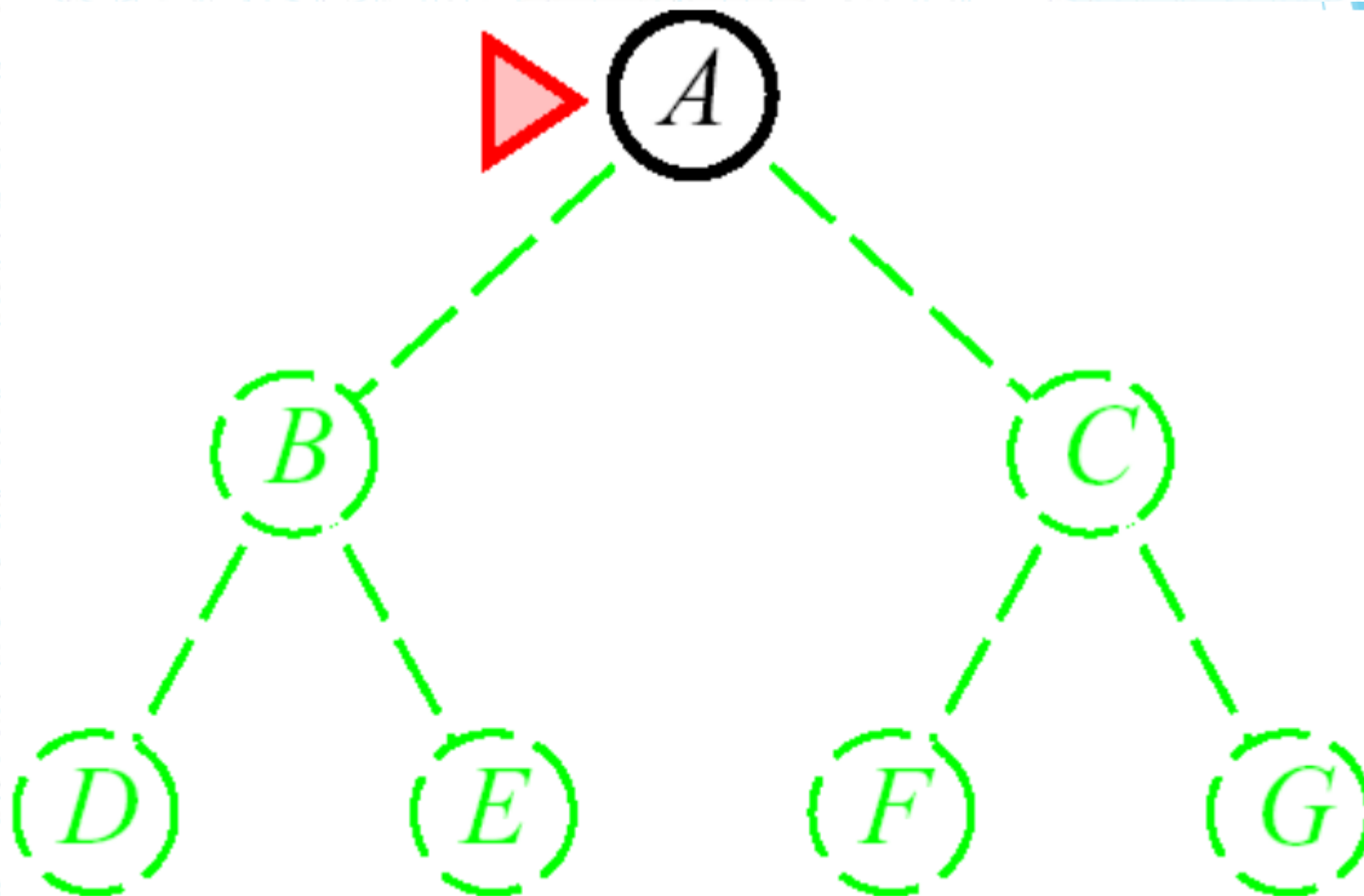
if GOAL-TEST[*problem*] applied to STATE(*node*) succeeds **then return** *node*

nodes $\leftarrow$ QUEUING-FN(*nodes*, EXPAND(*node*, OPERATORS[*problem*]))

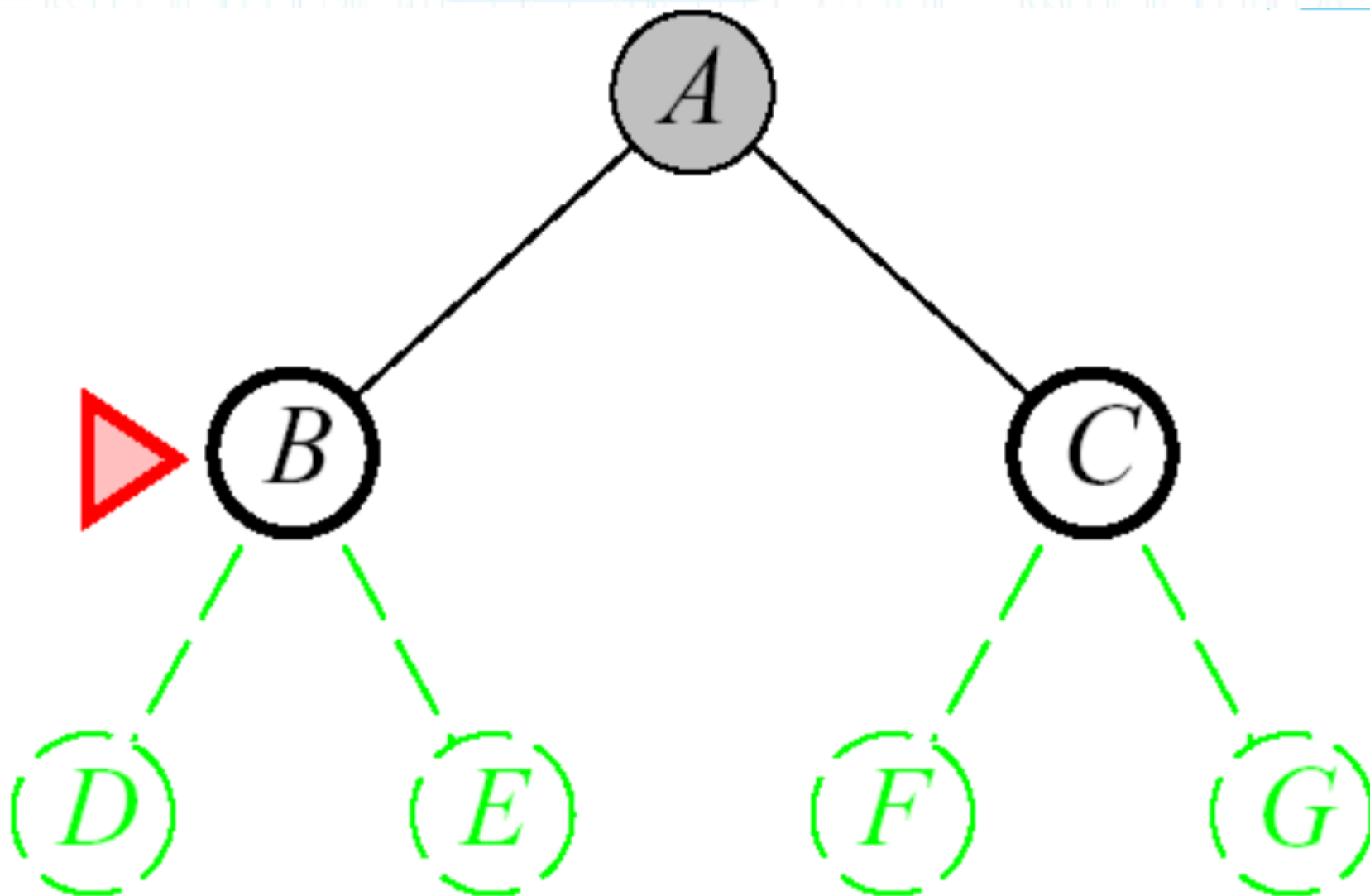
end

Өргөний хайлт (BFS)

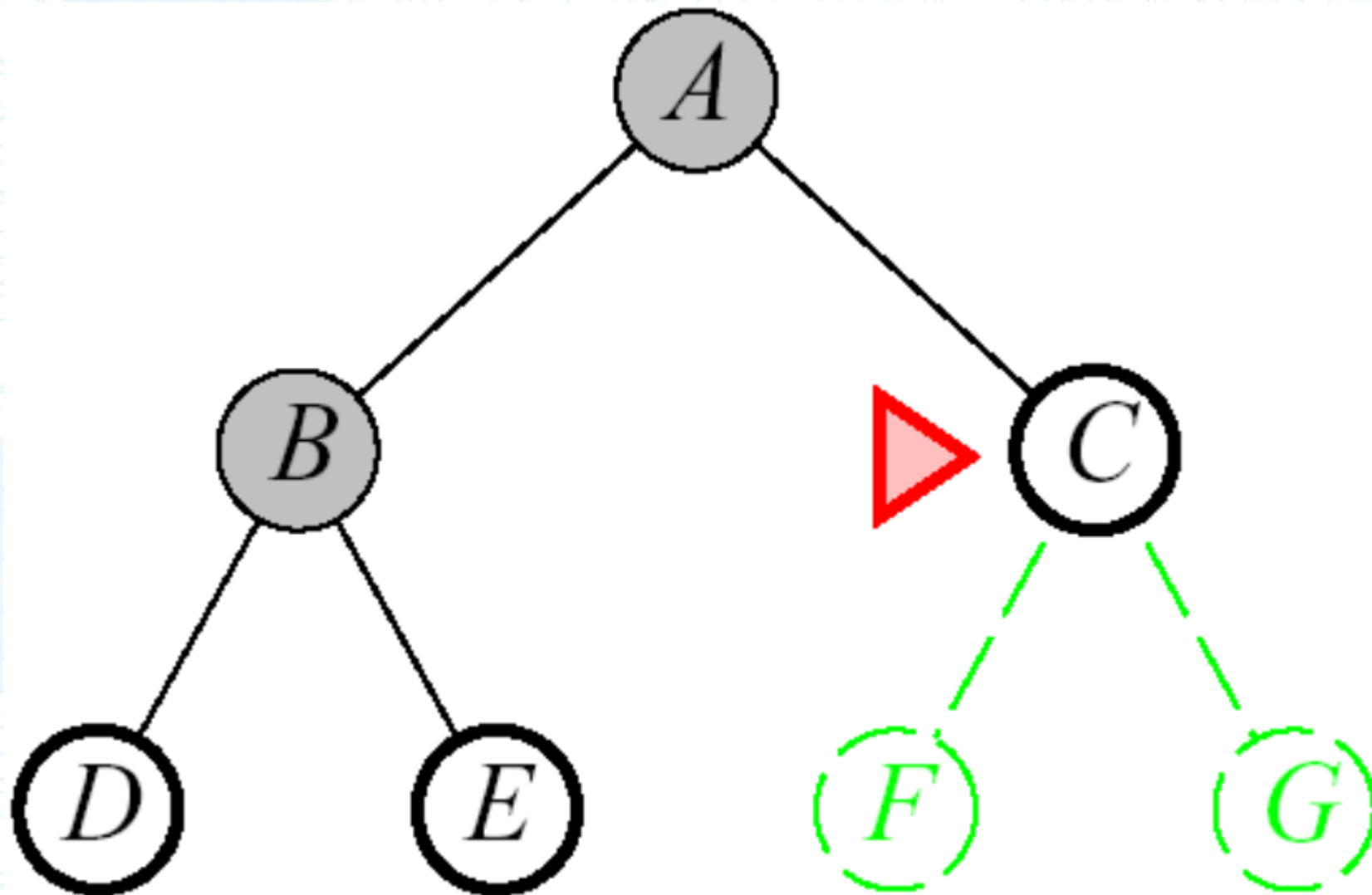
Queue=



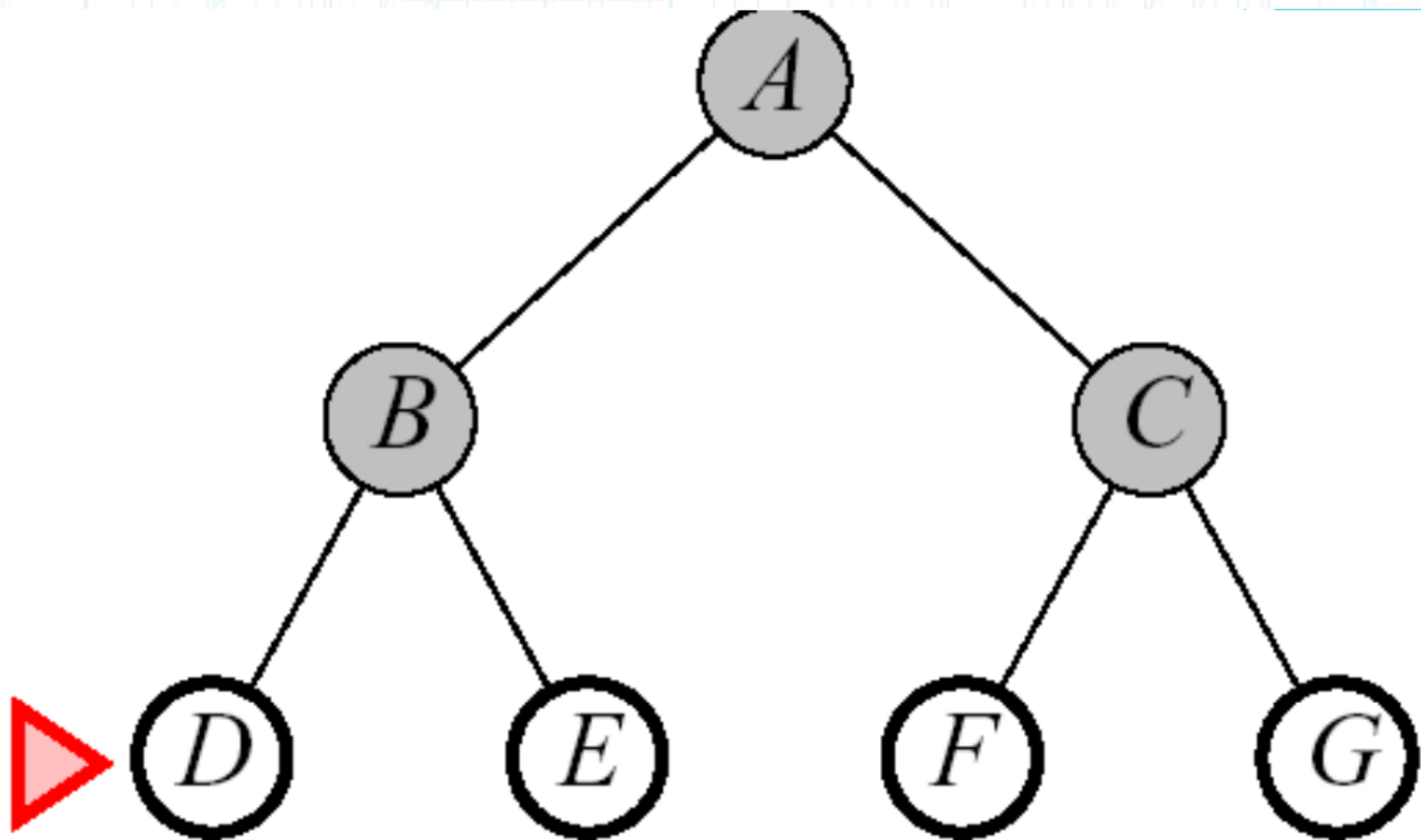
Өргөний хайлт (BFS)



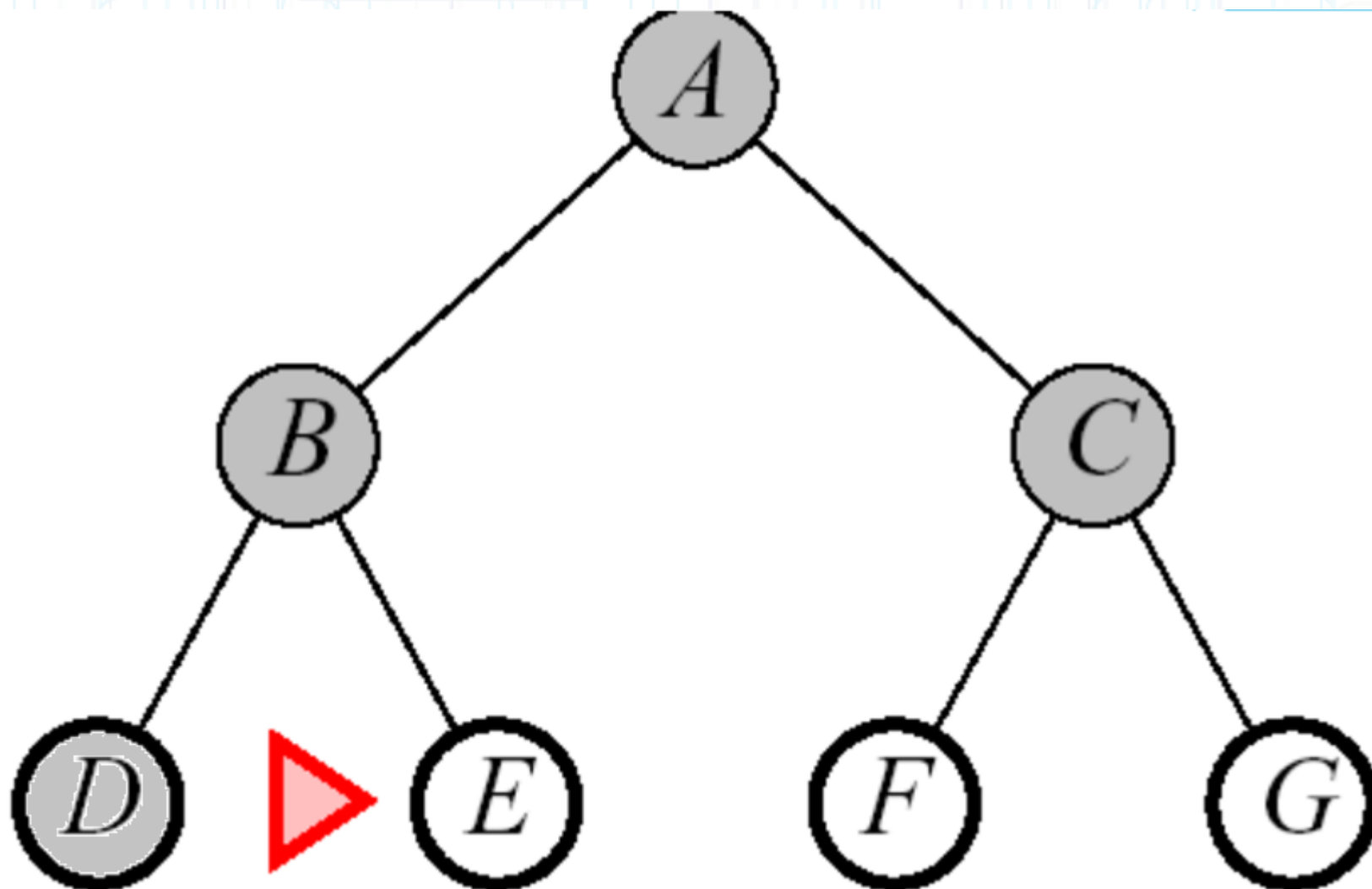
Өргөний хайлт (BFS)



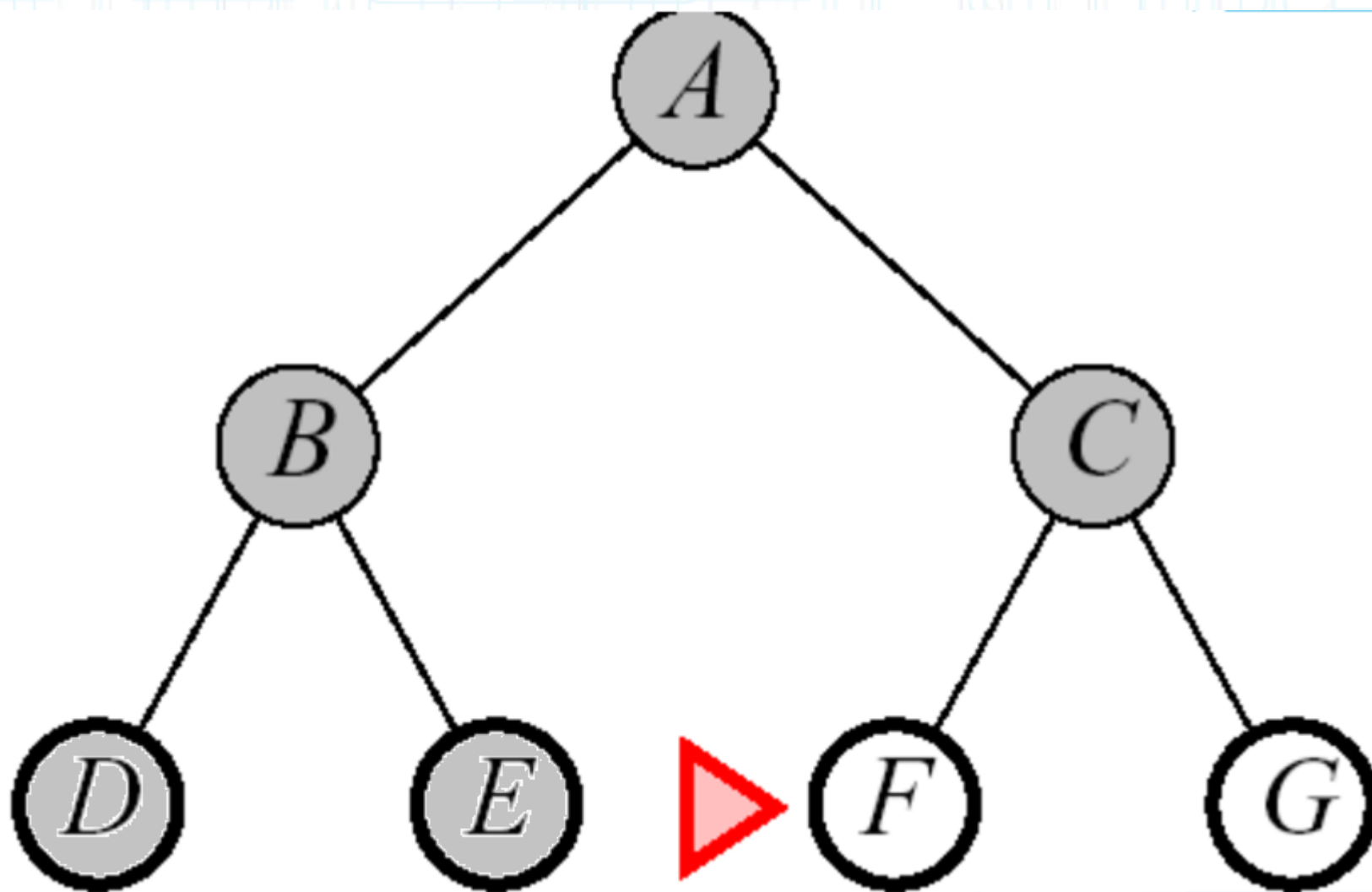
Өргөний хайлт (BFS)



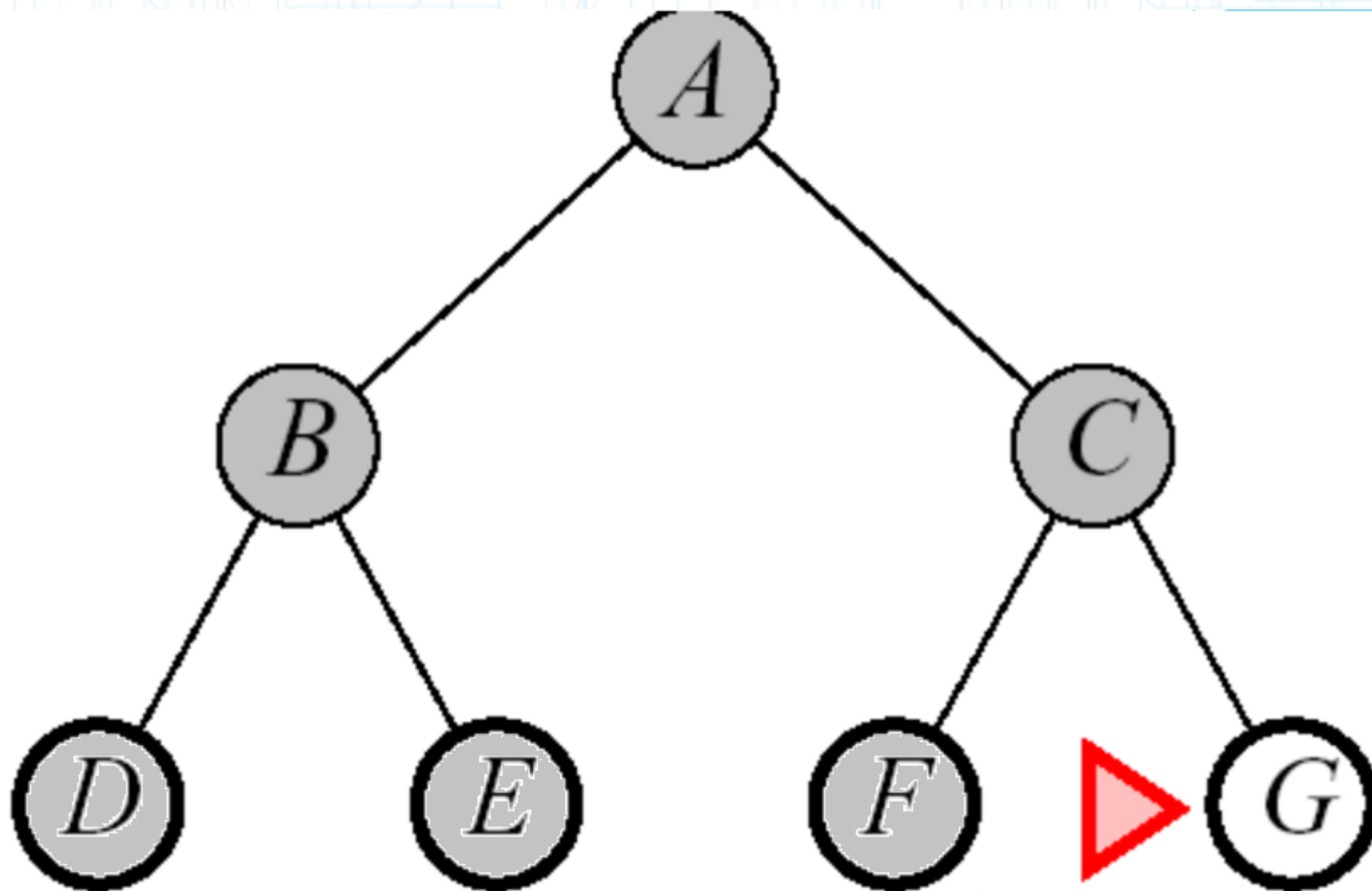
Өргөний хайлт (BFS)



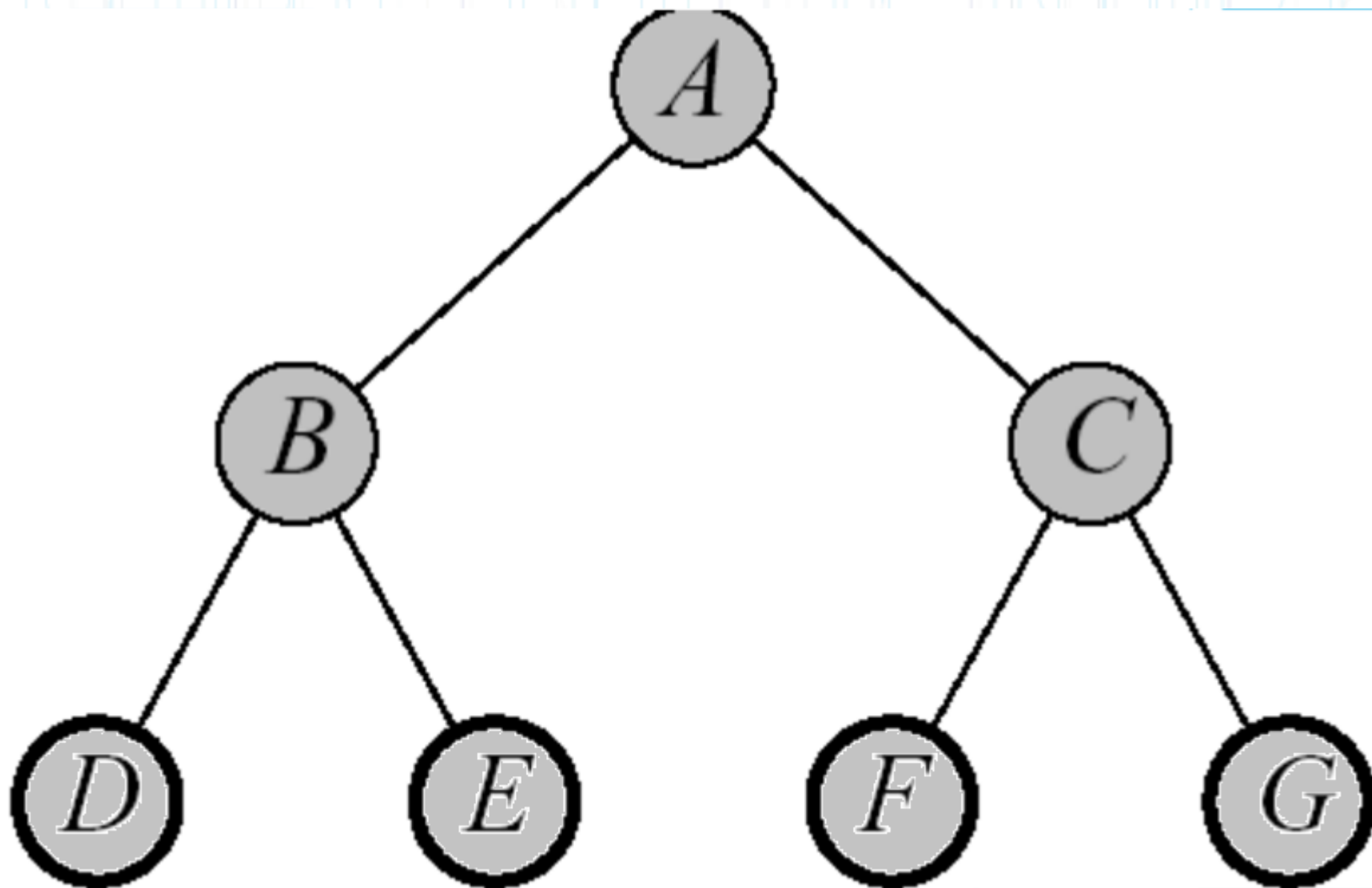
Өргөний хайлт (BFS)



Өргөний хайлт (BFS)



Өргөний хайлт (BFS)



Өргөний хайлт (BFS)

Төгсгөлөг: Тийм (хэрвээ b нь төгсгөлөг бол)

Хугацаа: $1 + b + b^2 + b^3 + \dots + b^d + b(b^d-1)$

Өөрөөр хэлбэл экспоненциал өсөлттэй байнг

Санах ой: $1 + b + b^2 + b^3 + \dots + b^d + b(b^d-1)$

Оновчтой эсэх: Тийм (хэрвээ алхам бүрийн зардал нь 1 бол)

Өргөний хайлт (BFS)

Set all nodes to "not visited";

```
q = new Queue();
```

```
q.enqueue(initial node);
```

```
while ( q ≠ empty ) do
```

```
{
```

```
  x = q.dequeue();
```

```
  if ( x has not been visited )
```

```
  {
```

```
    visited[x] = true;    // Visit node x !
```

```
    for ( every edge (x, y) /* we are using all edges ! */ )
```

```
      if ( y has not been visited )
```

```
        q.enqueue(y);    // Use the edge (x,y) !!!
```

```
    }
```

```
}
```

Өргөний хайлт (BFS)

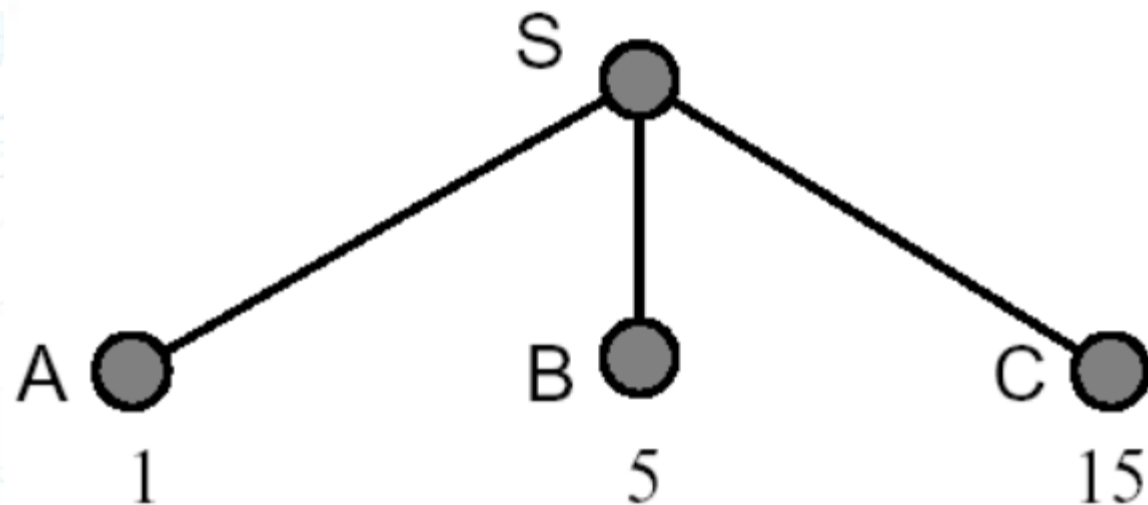
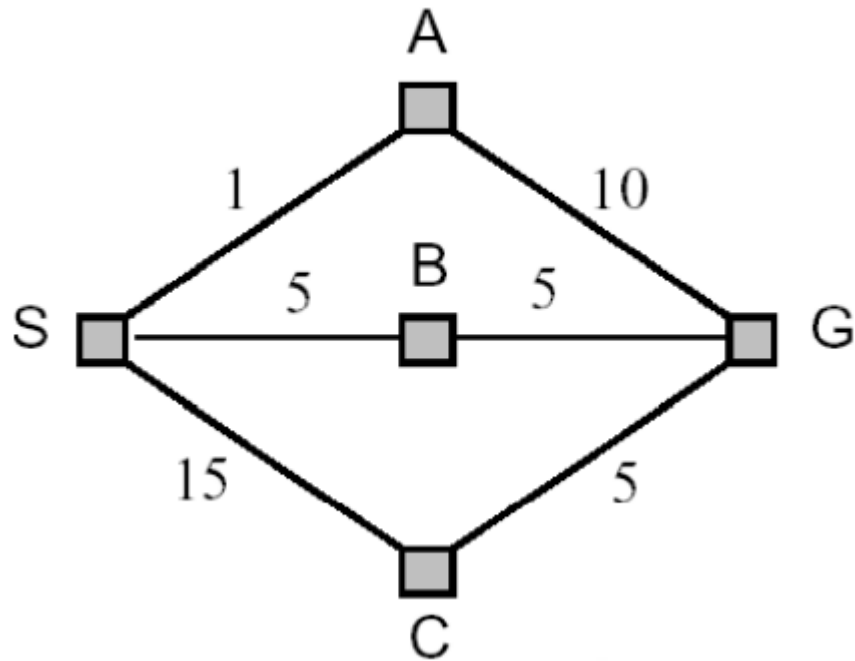
Depth	Nodes	Time	Memory
0	1	1 millisecond	100 bytes
2	111	.1 seconds	11 kilobytes
4	11,111	11 seconds	1 megabyte
6	10^6	18 minutes	111 megabytes
8	10^8	31 hours	11 gigabytes
10	10^{10}	128 days	1 terabyte
12	10^{12}	35 years	111 terabytes
14	10^{14}	3500 years	11,111 terabytes

$b = 10$ тохиолдолд 1000 nodes/sec; 100 bytes/node BFS хайлтад зарцуулагдах санах ойн үзүүлэлт!

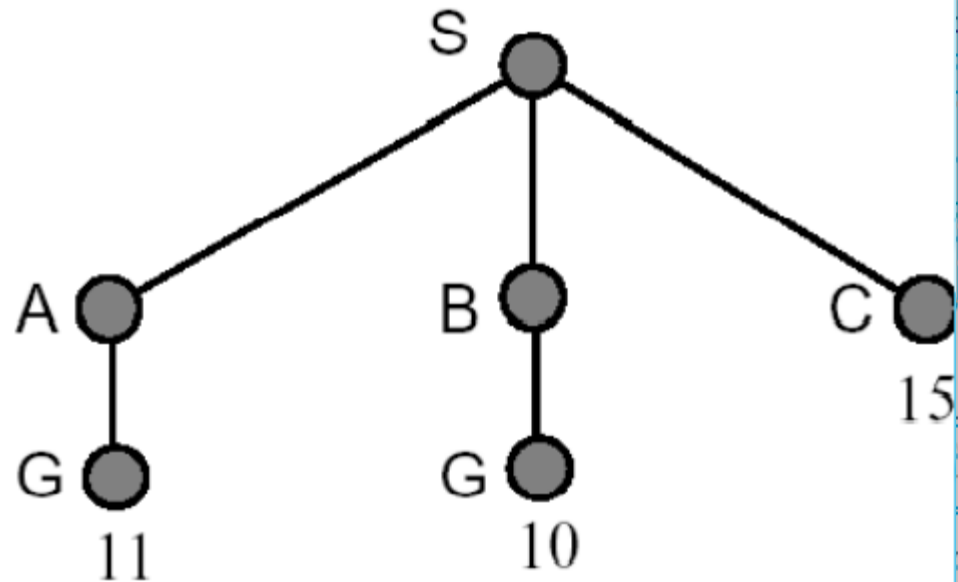
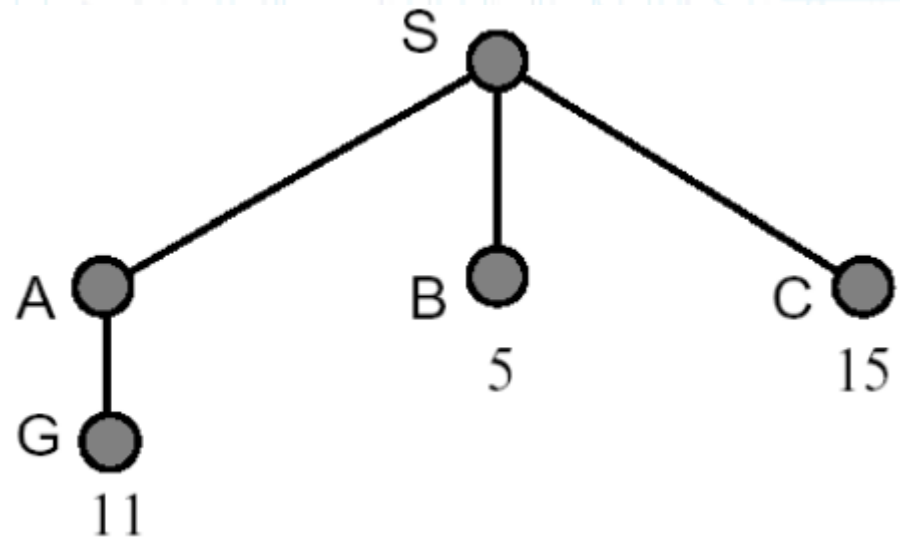
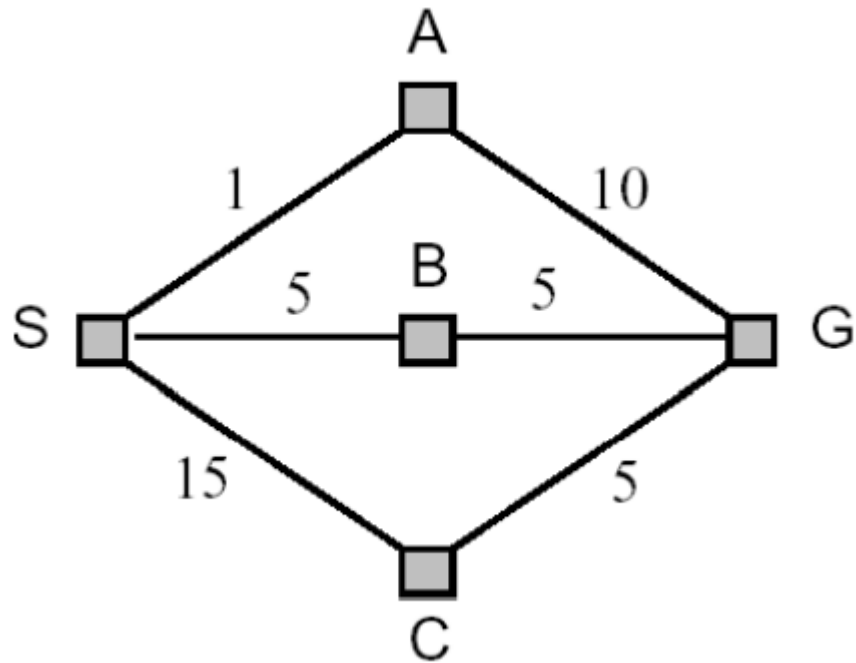
Өртөгөөр хайлт (UCS)

- Хамгийн бага зардалтай зангилааг эхэлж өргөтгөнө.
- $G(N)$ функц нь n анхны төлөвөөс N зангилаа хүртэлх замын зардал.
- BFS нь $G(n) = \text{Depth}(n)$ үеийн UCS хайлтын тухайн тохиолдол юм.

Өртөгөөр хайлт (UCS)



Өртөгөөр хайлт (UCS)



Өртөгөөр хайлт (UCS)

$$g(SUCCESSOR(n)) \geq g(n)$$

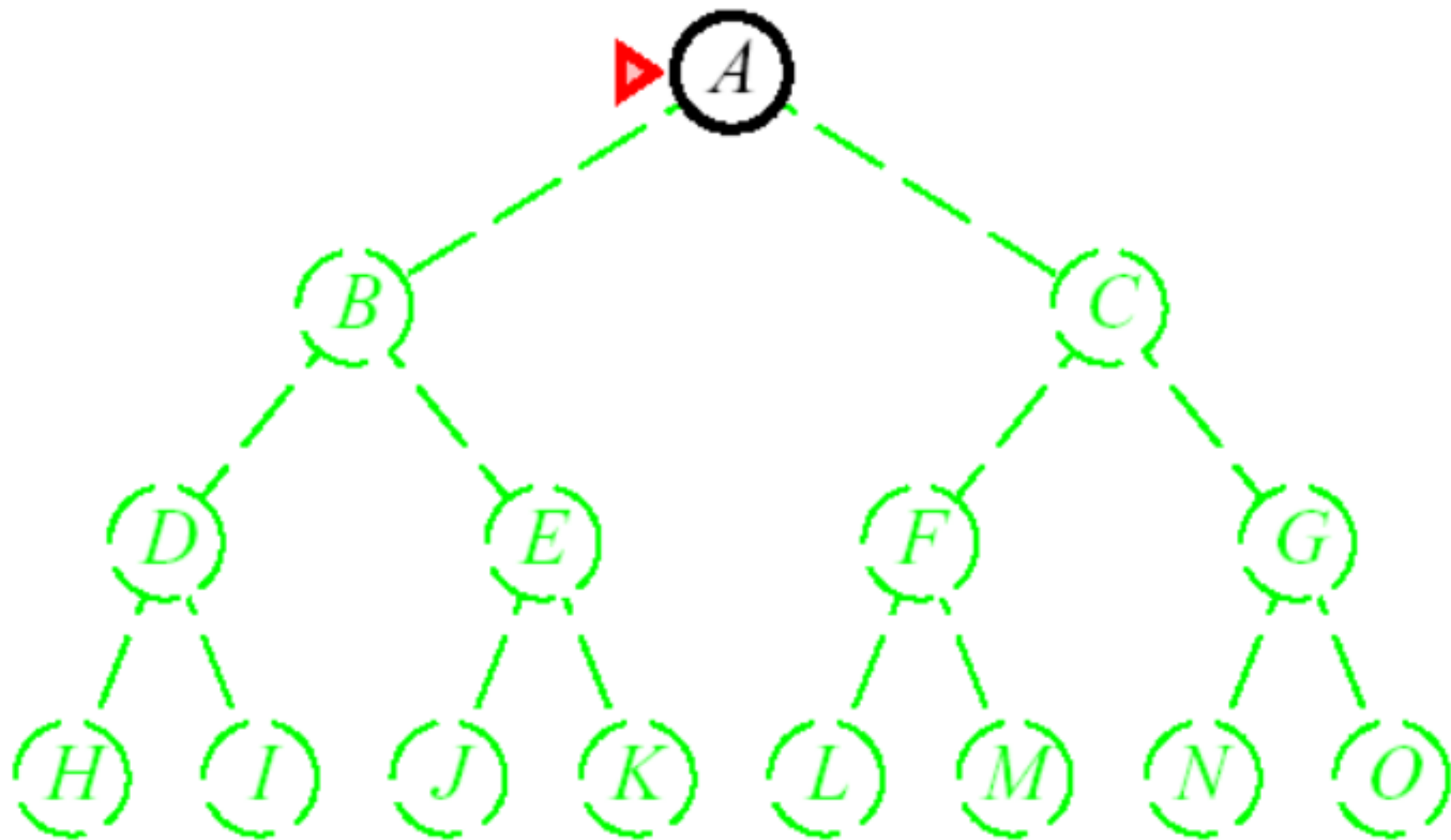
Нөхцөл биелж байвал UCS нь хамгийн бага зардалтай замыг хайж олж чадна.

Гүнээр хайлт (DFS)

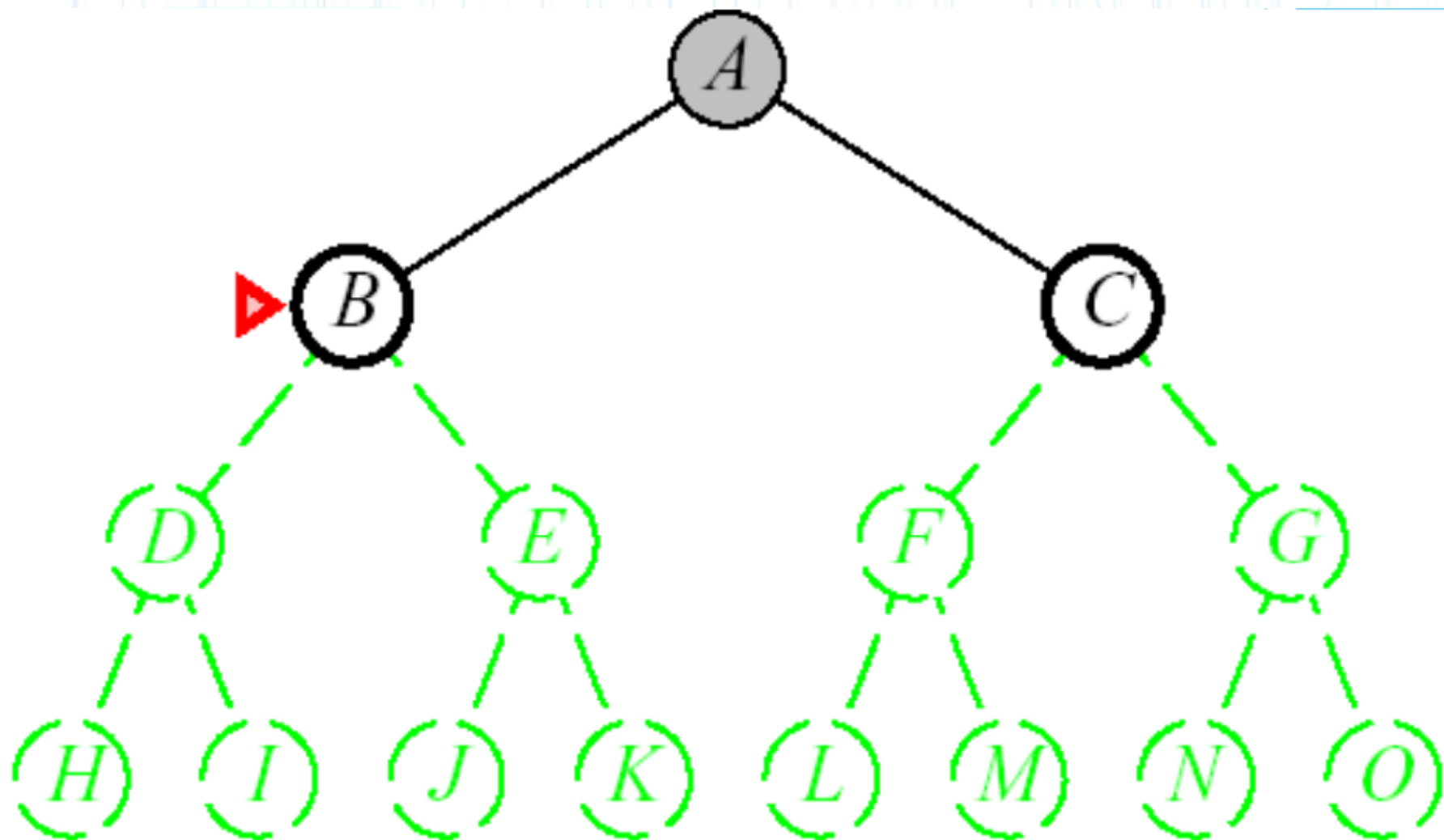
Санах ойг маш бага шаарддаг онцлогтой.
($b \cdot m$ / m : maximum depth/)

Дээр үзсэн жишээнд $d=12$ байхад BFS хайлт 111 терабайт санах ой шаарддаг бол DFS хайлт 12kb санах ой шаардана.

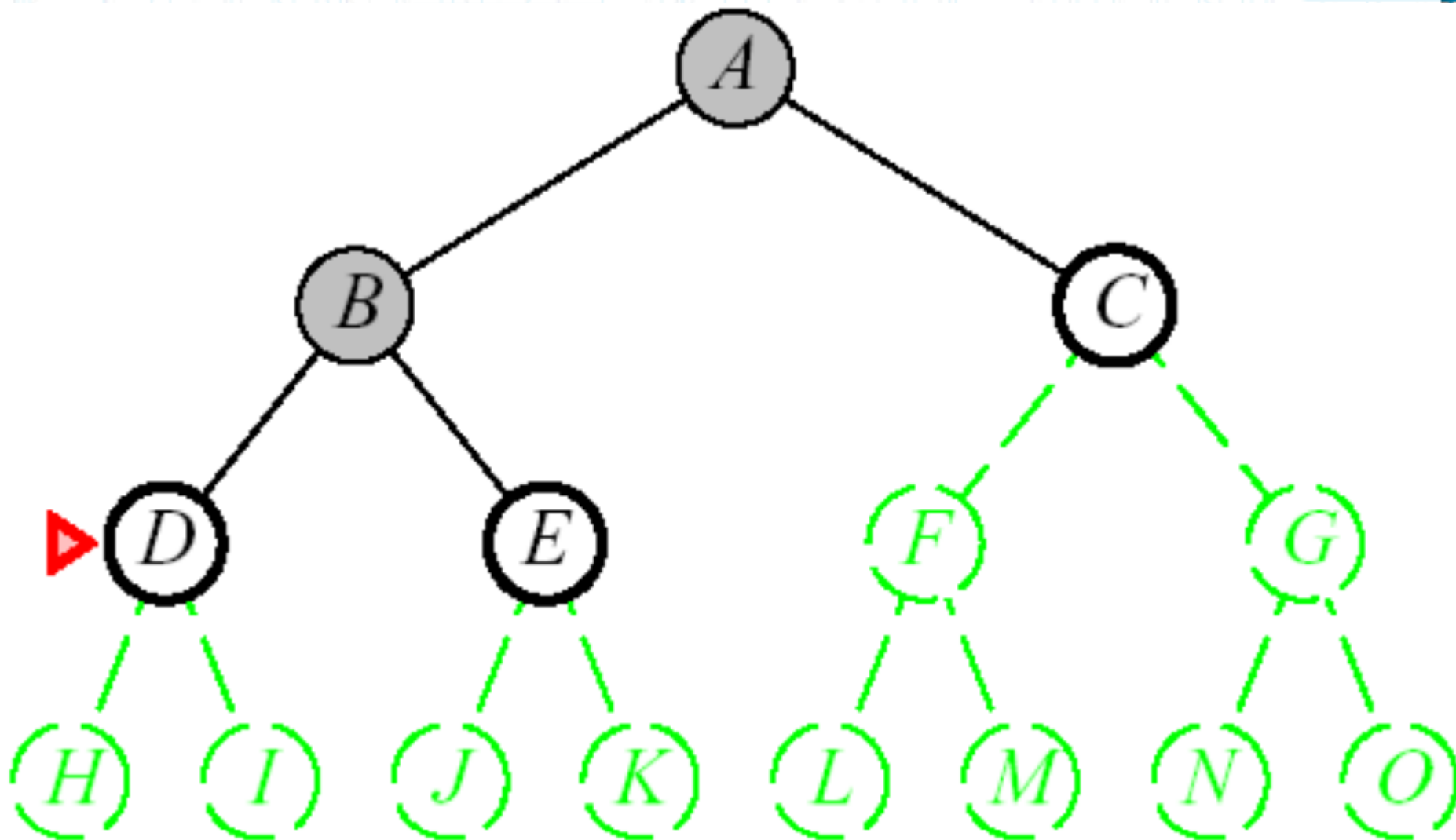
Гүнээр хайлт (DFS)



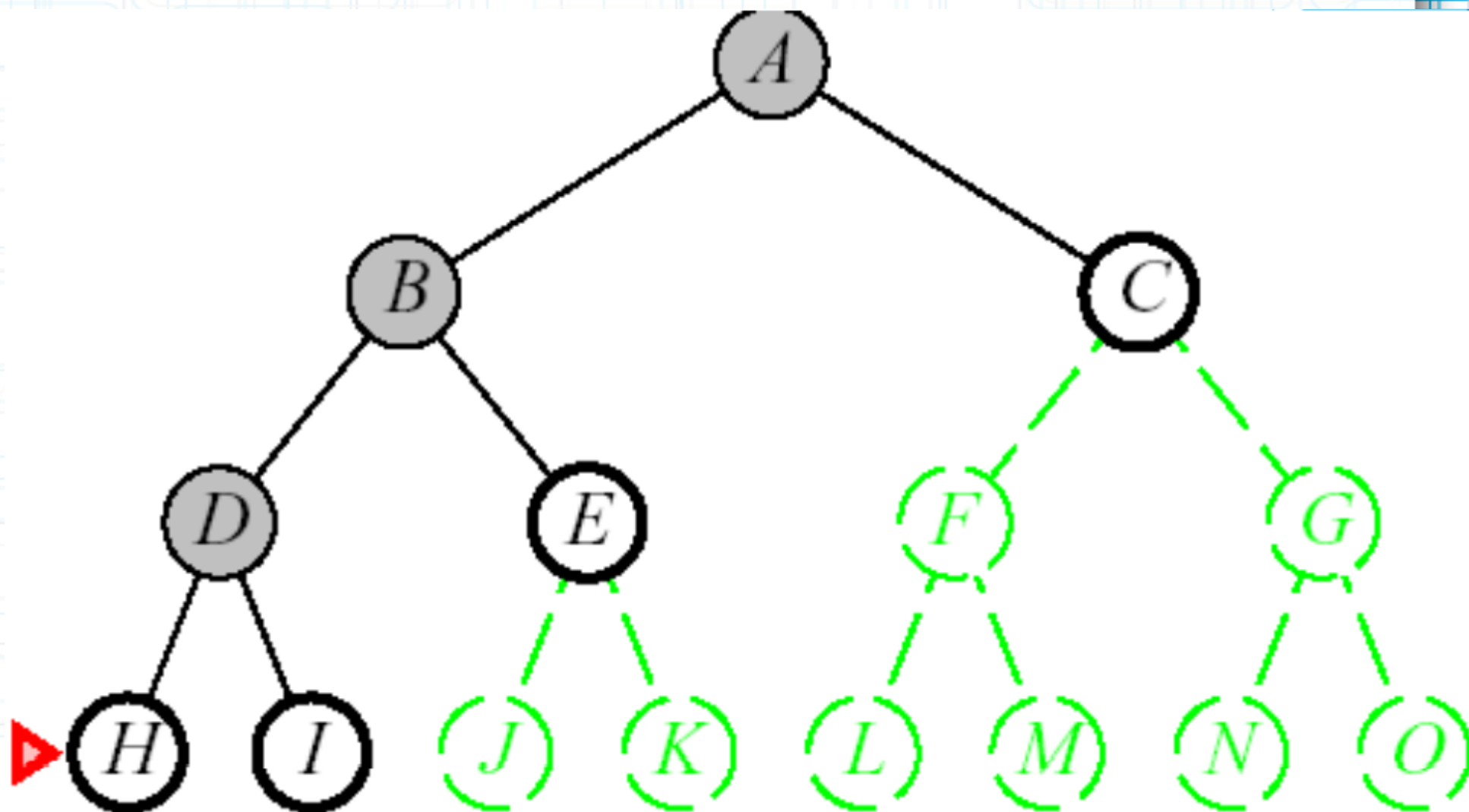
Гүнээр хайлт (DFS)



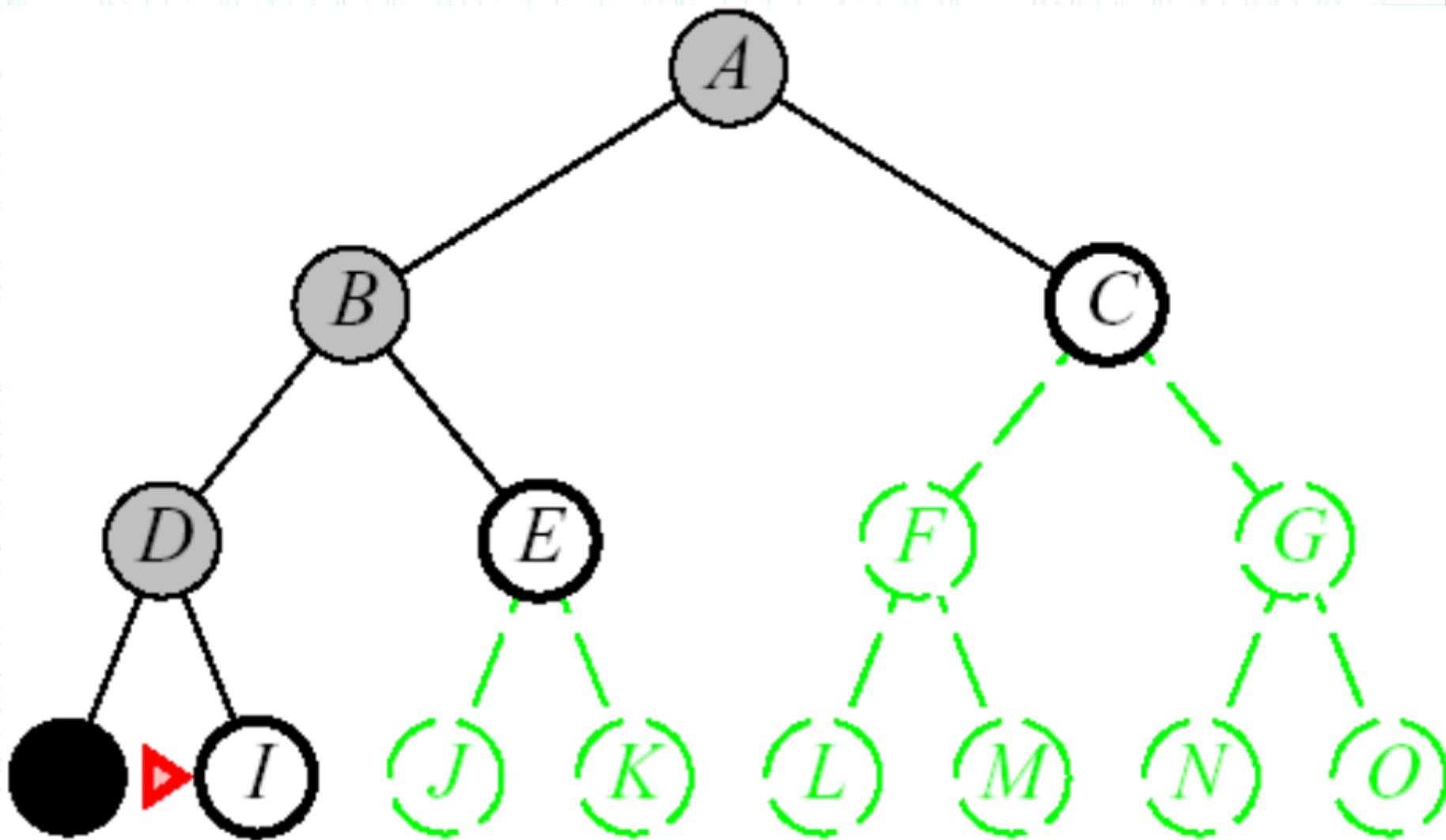
Гүнээр хайлт (DFS)



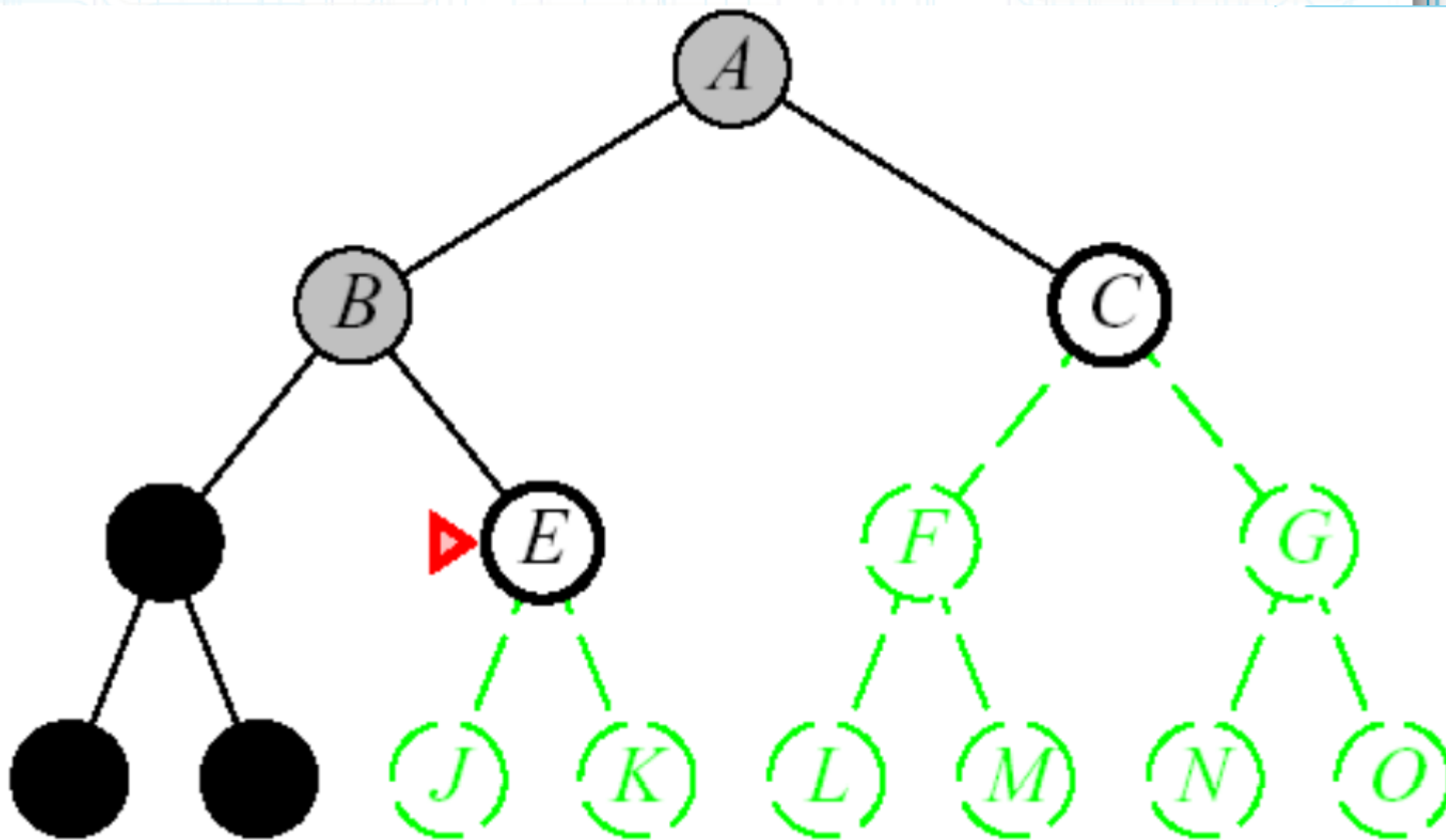
Гүнээр хайлт (DFS)



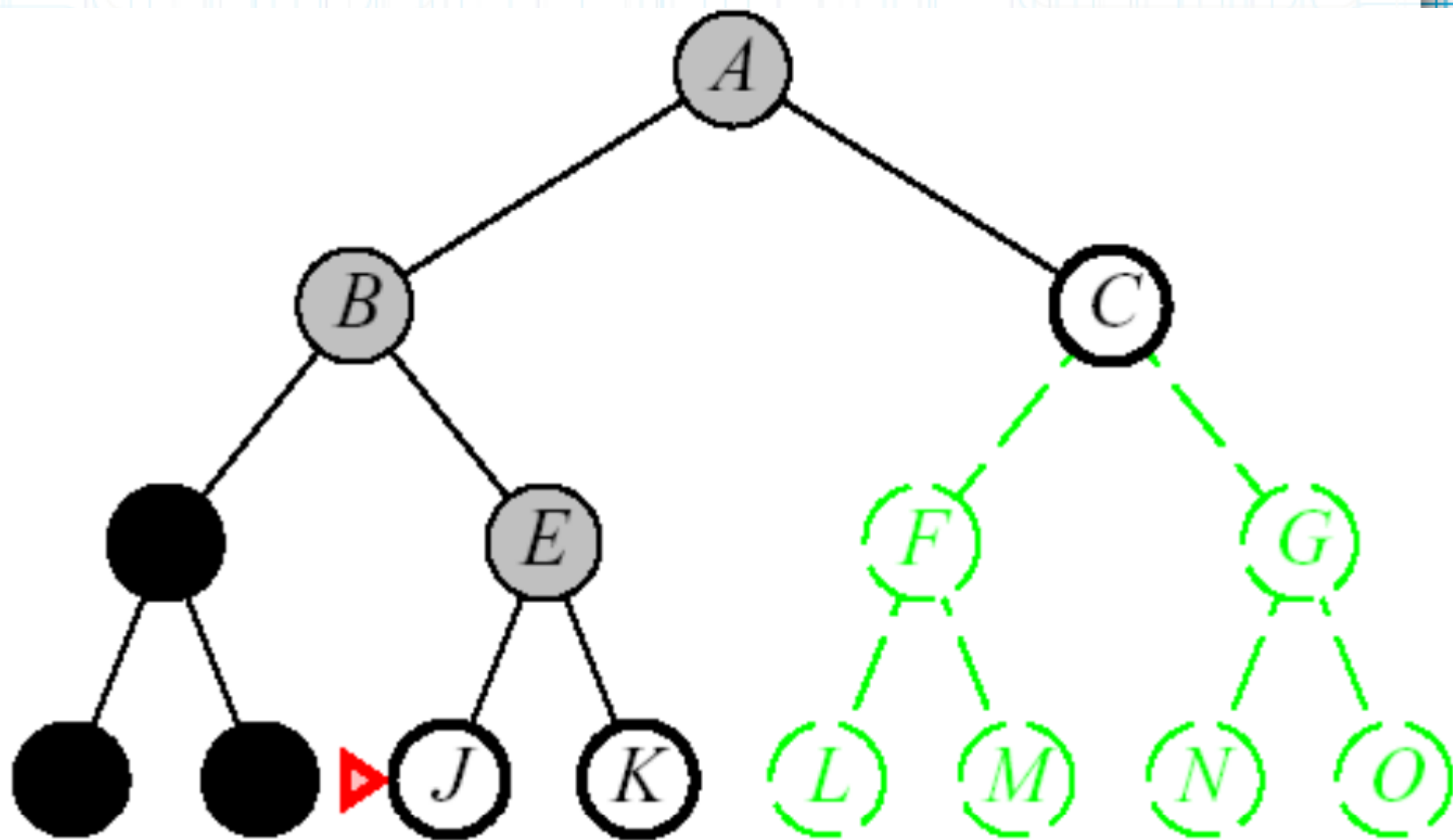
Гүнээр хайлт (DFS)



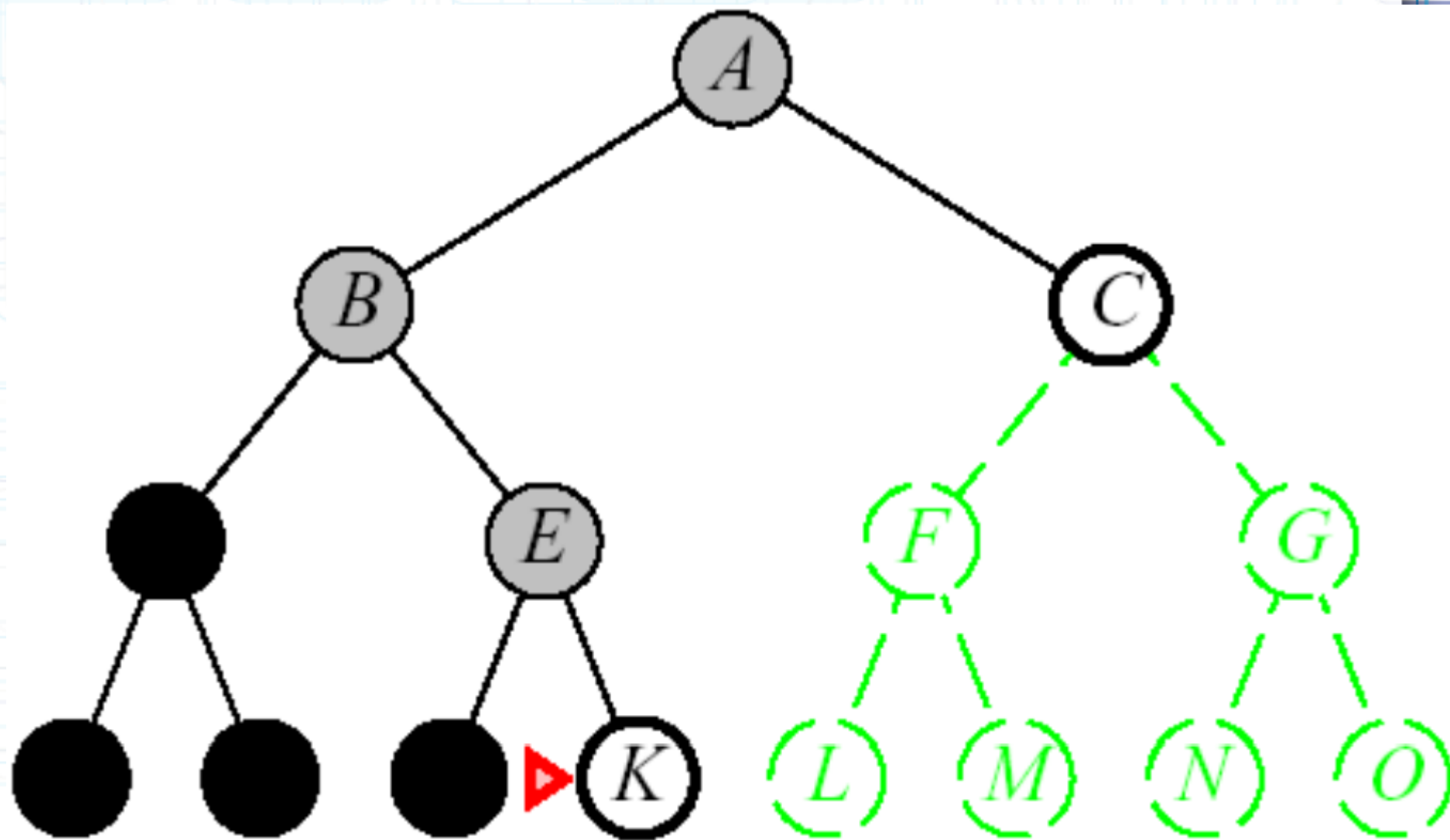
Гүнээр хайлт (DFS)



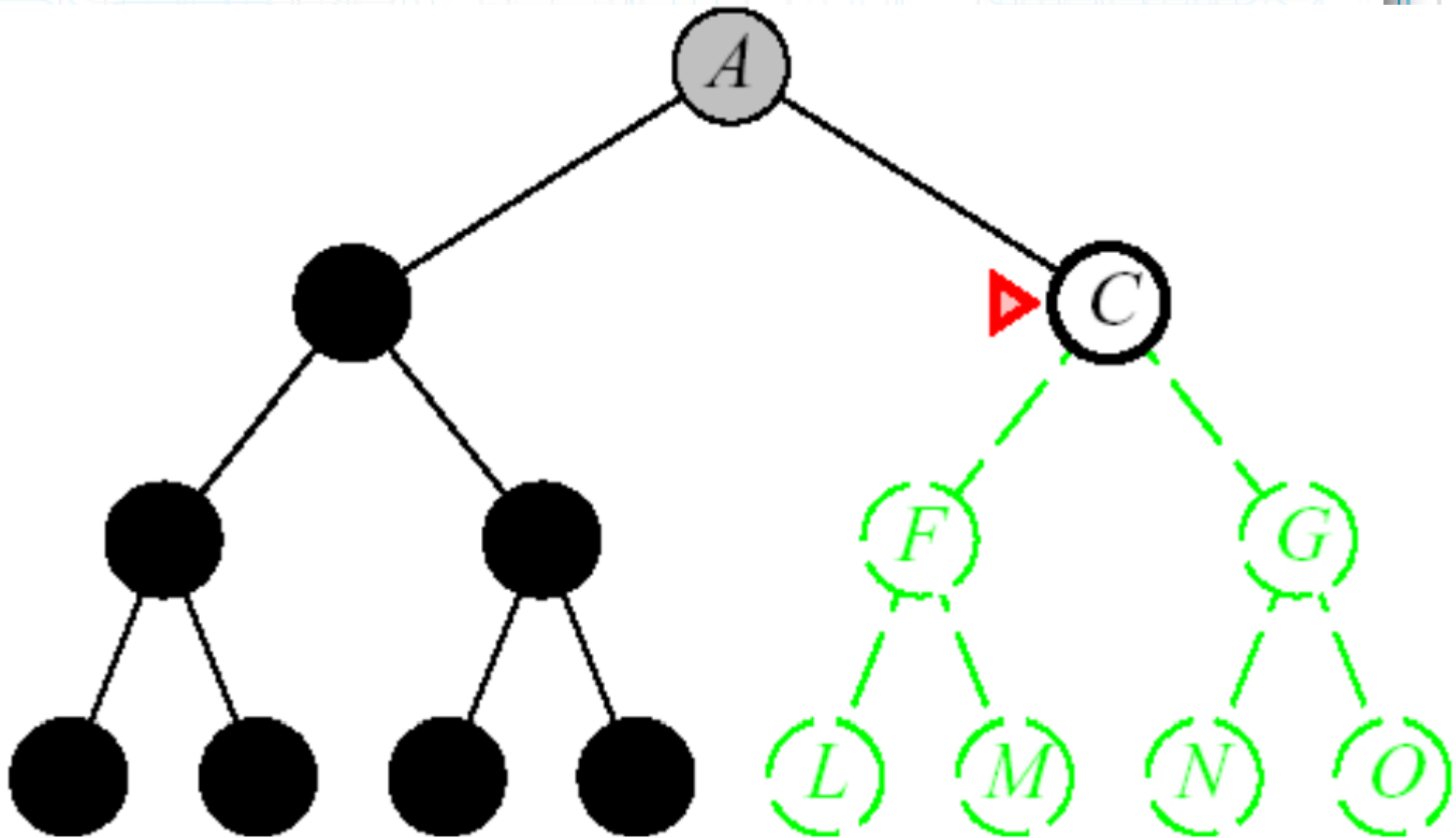
Гүнээр хайлт (DFS)



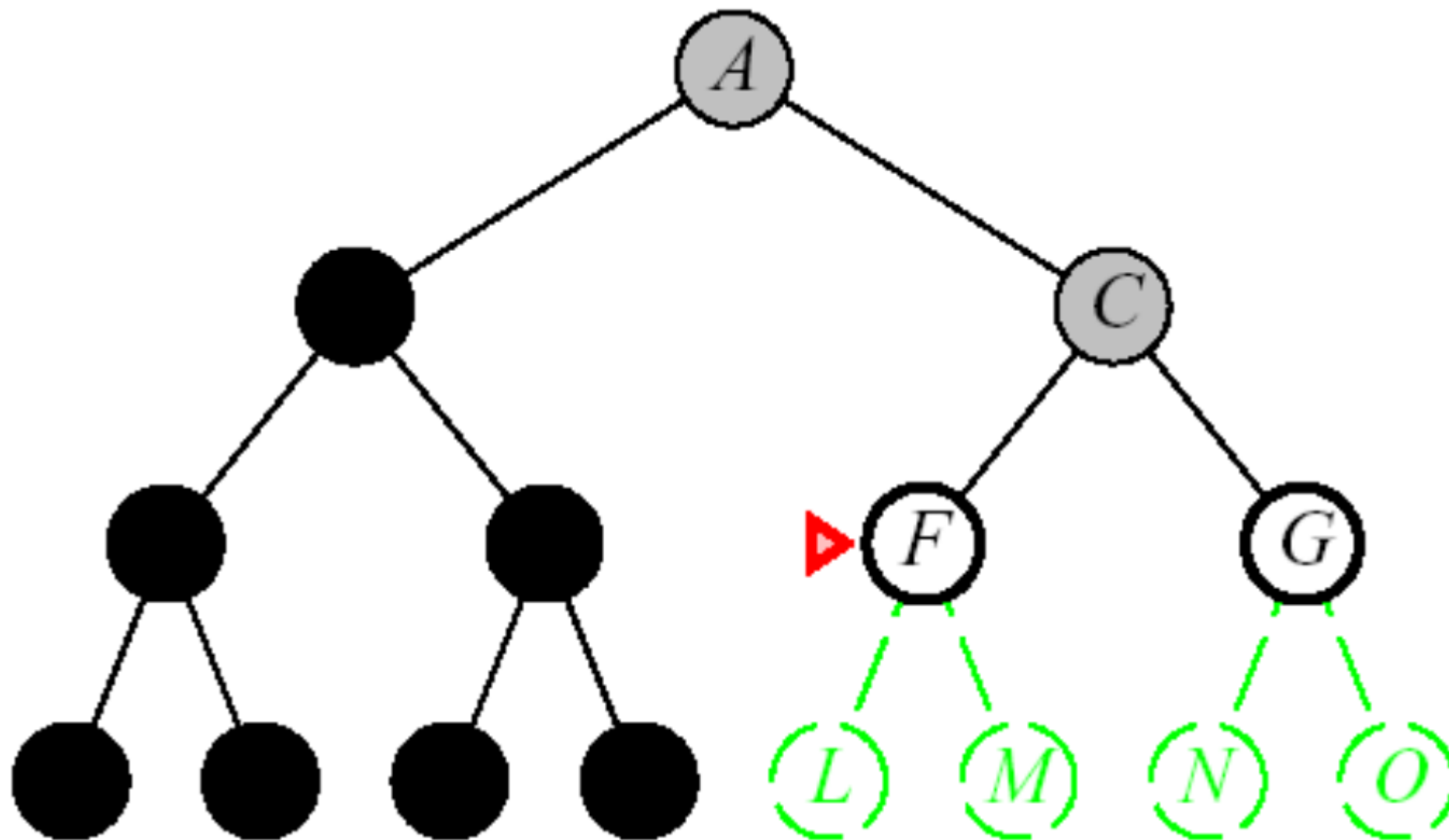
Гүнээр хайлт (DFS)



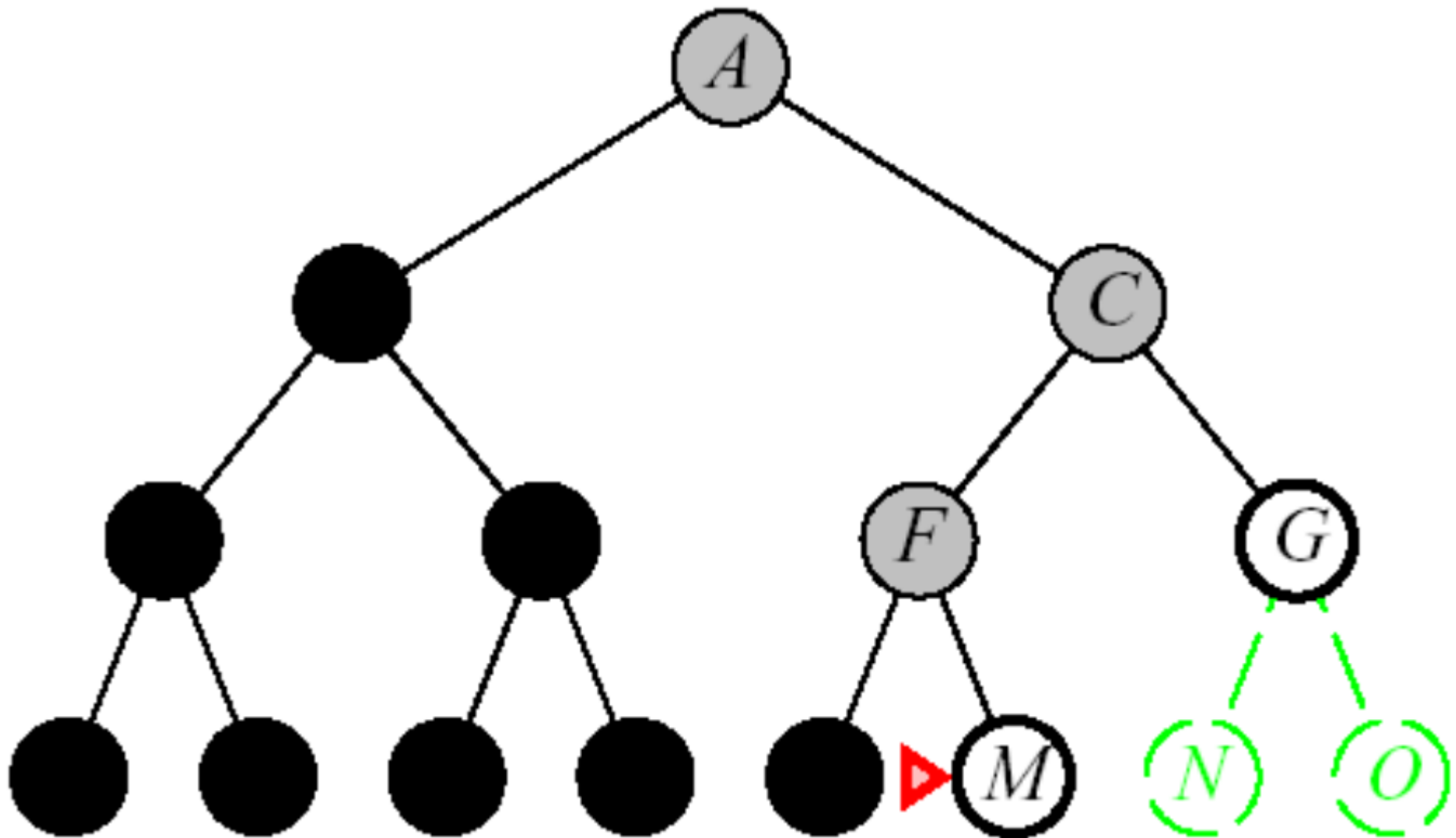
Гүнээр хайлт (DFS)



Гүнээр хайлт (DFS)



Гүнээр хайлт (DFS)



Гүнээр хайлт (DFS)

- Хэрвээ тухайн асуудлын шийдэл нь олон байдаг бол BFS ээс илүү хурдан шийд олох магадлалтай.
- Модны дээд хэсэгт шийд оршин байхад түүнийг олохгүй буруу замаар ороод сураггүй болж болзошгүй хайлт...

Гүнээр хязгаарласан хайлт (DLS)

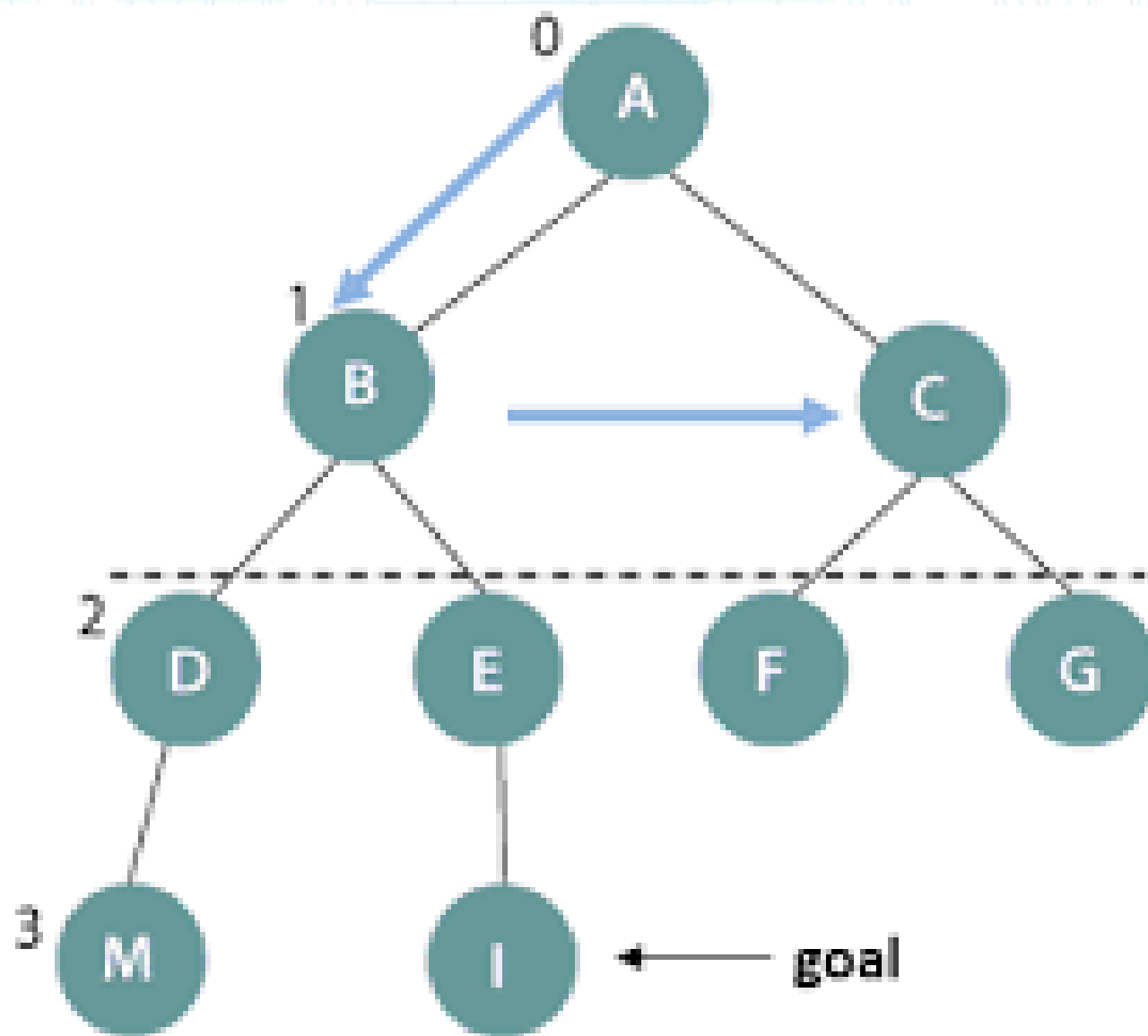
- BFS хайлтын максимум M гүнийг оптимал L гүнээр хязгаарлан хайх хайлт юм.
- Гэхдээ үүний тулд тухайн асуудлын оптимал гүнг (шийд баталгаатай оршин байх хамгийн бага гүн) хүн тооцоолон олсон байх хэрэгтэй.

Гүнээр хязгаарласан хайлт (DLS)

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns soln/fail/cutoff  
  RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)
```

```
function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff  
  cutoff-occurred?  $\leftarrow$  false  
  if GOAL-TEST[problem](STATE[node]) then return node  
  else if DEPTH[node] = limit then return cutoff  
  else for each successor in EXPAND(node, problem) do  
    result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)  
    if result = cutoff then cutoff-occurred?  $\leftarrow$  true  
    else if result  $\neq$  failure then return result  
  if cutoff-occurred? then return cutoff else return failure
```

Гүнээр хязгаарласан хайлт (DLS)



Давтамжтай гүний хайлт(IDS)

- Хайлтын гүнийг 0-с эхлэн аажмаар нэмэгдүүлэн хайх хайлт.
- Ямар ч асуудлын оптимал гүнг өөрөө олох давуу талтай

Давтамжтай гүний хайлт(IDS)

- Цагт хавчигдсан системд хэрэглэж болно. Жишээ нь цагтай шатар. Өгөгдсөн цагт тохируулан шийдийг олох чадвартай.
- DFS хайлтыг бодвол 11% илүү тооны зангилаа өргөтгөх тул арай илүү удаан байж болох...

Давтамжтай гүний хайлт(IDS)

Iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution
  inputs: problem, a problem
  for depth  $\leftarrow$  0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
  end
```

Давтамжтай гүний хайлт(IDS)

Iterative deepening search $l = 0$

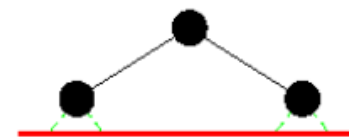
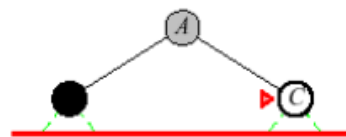
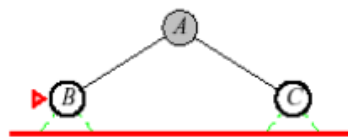
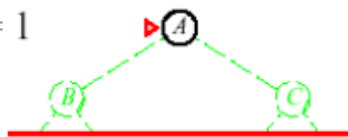
Limit = 0



Давтамжтай гүний хайлт(IDS)

Iterative deepening search $l = 1$

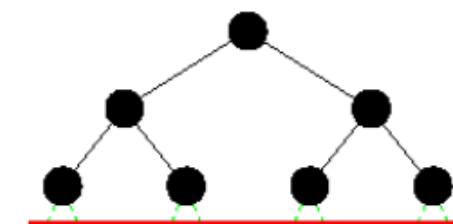
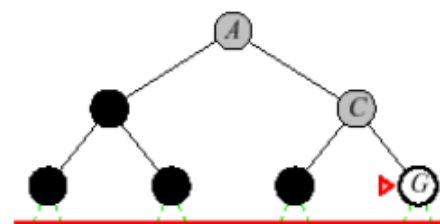
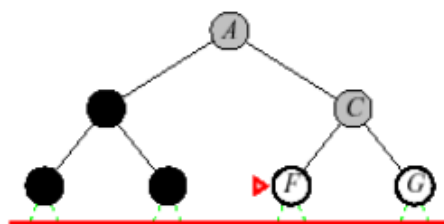
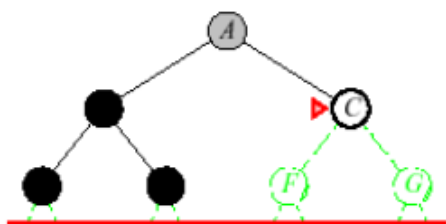
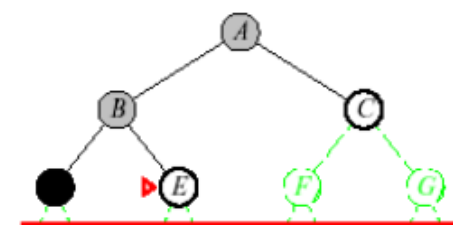
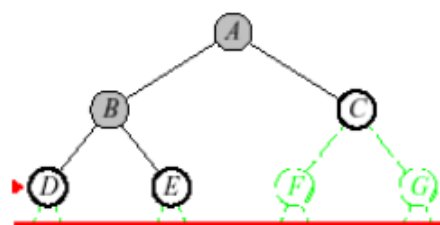
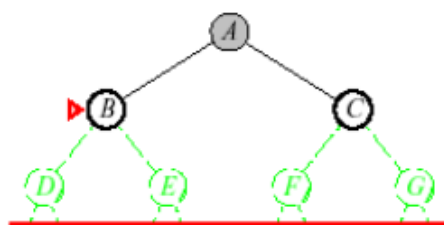
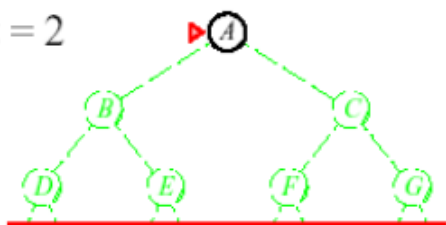
Limit = 1



Давтамжтай гүний хайлт(IDS)

Iterative deepening search $l = 2$

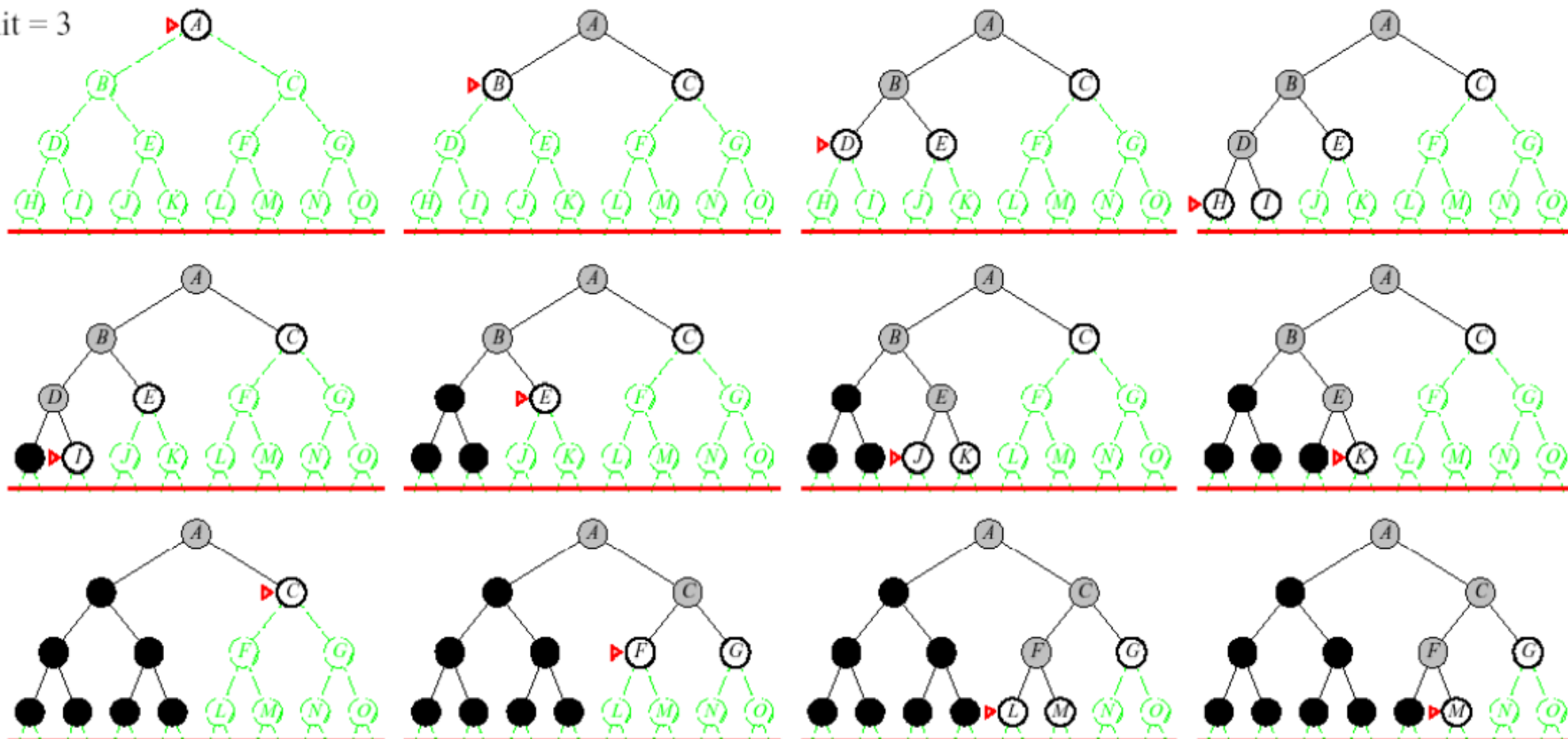
Limit = 2



Давтамжтай гүний хайлт (IDS)

Iterative deepening search $l = 3$

Limit = 3



Bidirectional Search (BS)

- Анхны төлөв болон зорилгын төлөв хоёроос нь өөд өөдөөс нь хайна. Хоёр хайлт огтлолцох үед шийд олдоно.
- “Хариунаас нь бодох” аргатай төстэй...

